



Soft_gAming

18 часов назад

Один движок на два рантайма: как мой сайт на Next.js делит формулы с Flutter-приложением



16 мин



4.8K

Dart*, Flutter*, HTML*, JavaScript*, Node.JS*

Мнение

Прошлой зимой я писал [тут](#) про «Мастерок» — строительный калькулятор на Flutter для RuStore. Приложение поехало в прод, набрало 4.9 звезды, и в какой-то момент пришло осознание: аудитория смартфонных приложений — это аудитория смартфонных приложений. А человек, который в обед нагуглил «сколько мешков ротбанда на 20 квадратов», в магазин приложений не полезет. Он хочет страницу в браузере. Желательно без куки-баннера на полэкрана, без интерфейса из 2012-го и без того, чтобы перед ответом на вопрос ему предлагали посмотреть пять реклам.

Так появился `getmasterok.ru` — веб-половина той же экосистемы. Сайт на Next.js 15, шестьдесят один калькулятор, ИИ-прораб, SEO, блог, всё как положено. И с одним неочевидным вызовом, который стал главным сюжетом этой статьи.

А вызов я нашёл не сам. Его нашёл пользователь.

В личку прилетело сообщение, которое я до сих пор держу в избранном:

«У вас на сайте под один столб забора из профлиста нужно 0,04 м³ бетона. А в вашем же приложении — 0,055. Где правда?»

Я сел проверять. Правда оказалась такой: на сайте я считал яму под столб по диаметру 100 мм, а в приложении — по 150 мм. В одном месте константа, в другом константа, и никто этого не замечал, пока пользователь не открыл обе поверхности рядом.

Это был не баг. Это был диагноз.

Сайт и приложение — одна экосистема, один бренд, один пользователь, и он имеет полное право ожидать, что на двух моих поверхностях цифры совпадут. А у меня, по сути, было два независимых проекта, которые считали «примерно то же самое» двумя независимыми кодовыми базами на двух разных языках. Один на Dart, второй на TypeScript. Мантра о DRY в экосистеме было равно ноль.

С этого момента я начал перестраивать оба проекта под идею единого источника истины. Но сначала — про сам сайт и про то, зачем он вообще понадобился рядом с уже живущим приложением.

Зачем ещё и сайт, если уже есть приложение

Когда я писал Flutter-версию, я смотрел на «Мастерок» как на мобильный инструмент для прораба: открыл на стройке, ввёл цифры, получил результат, закрыл. Оно до сих пор так и работает, и

аудитория у него своя — люди, которым удобно иметь калькуляторы под рукой в виде иконки на домашнем экране.

Но чем дольше я за ним наблюдал, тем яснее становилось: основной сценарий поиска у русскоязычного строительного пользователя сегодня — это не магазин приложений. Это Яндекс и Google. Человек начинает с запроса «сколько клея на плитку 60х60», получает десять страниц выдачи, открывает первую, вторую, третью, сравнивает цифры, выбирает ту, где ответ выглядит наиболее вменяемо. И уходит. Ставить приложение ради одного расчёта он не будет, это слишком высокая цена за одноразовую задачу.

Поэтому сайт — это не замена приложению, а его веб-проекция. Приложение для тех, кто работает со стройкой регулярно. Сайт для всех остальных, и этих «всех остальных» в интернете сильно больше.

Если честно говорить про долгосрочную перспективу, то у сайта потолок роста выше. Поисковый трафик масштабируется предсказуемо: больше калькуляторов, будет больше посадочных страниц, больше страниц, соответственно больше мест в выдаче. Приложение же упирается в свою собственную воронку: публикация в сторе, попадание в топ категории, отзывы. И это не плохо, просто это другой рынок. Поэтому Flutter-приложение я ни в коем случае не бросаю: там своя живая база пользователей, свой фидбэк и свои задачи, которые на вебе не решаются (офлайн, QR-шеринг расчётов, сохранённые проекты). Но если бы меня сейчас спросили, куда я вложу следующий год разработки, конечно я бы честно ответил: в веб. Там больше возможностей и можно разгуляться.

Стек

Под капотом ничего экзотического. Фронт — Next.js 15.5 с App Router в режиме SSR. Именно SSR через `next start`, а не `static export`, и почему, это отдельная история, до которой доберёмся. Стилизация на Tailwind 4.2 (новый движок на PostCSS, который наконец перестал требовать отдельный `tailwind.config.js`), иконки — lucide-react, небольшой 3D в отдельных калькуляторах плитки, стен и кровли — three.js.

Когда пользователь решает экспортировать расчёт в PDF или Excel, библиотеки jsPDF и xlsx подгружаются динамическим импортом, а не летят в основной бандл. Причина меркантильная: кнопкой «Скачать PDF» пользуется один человек из двадцати, а тащить ради него в каждый открытый калькулятор по паре мегабайт, так себе сделка.

Бэкенда как отдельного сервиса у меня нет. Всё живёт внутри Next.js в виде API-роутов на Node.js. Главный из них — `/api/mikhalych`, серверный прокси к OpenRouter для ИИ-чата. Про него отдельно расскажу ниже, включая историю, из-за которой этот прокси вообще появился.

Рейт-лимитер для Михалыча сделан максимально в лоб: обычная in-memory Map, двадцать запросов в минуту на IP. Я прекрасно знаю, что она сбрасывается на рестарте контейнера и сломается при масштабировании на несколько инстансов. На моих текущих объёмах трафика Redis это оверкилл уровня ИИ-сингулярности, и когда в комментариях в прошлый раз мне это говорили, я соглашался, но не делал. И до сих пор не делаю.

Из инфраструктуры всё стоит на Timeweb Cloud App Platform (Backend App на Node.js 24, авто-деплой от пуша в main). CI - GitHub Actions: линтер, vitest, build. Без зелёного CI в main не попадает ничего. Домен, HTTPS, HSTS preload, там, где им и положено быть. Блог живёт на отдельном self-hosted Ghost, как я его использую и зачем, тоже будет ниже.

Это всё имеет смысл упоминать только потому, что на этом стеке дальше и построен главный сюжет статьи.

Главный сюжет: единый источник истины на JSON

У меня был выбор. Либо раз в месяц ловить сообщения вида «а почему у вас по профлисту одна цифра, а у вас другая», либо перестать считать формулы в двух местах.

Я выбрал второе.

Идея простая: источник истины декларативный. Код формулы есть в обоих рантаймах, он короткий и однотипный. А вот всё, что могло разойтись: коэффициенты, упаковки, запасы, округления, шаги крепежа, минимальные толщины слоёв — всё это вынесено в отдельные JSON-спеки, по одной на калькулятор. Внутри такой спеки лежат, условно говоря, «правила игры» конкретного расчёта: какой диаметр лунки под столб считаем по умолчанию, сколько процентов брать на технологические потери, как округлять объём (в большую сторону всегда, в меньшую никогда), какой фасовкой продаётся материал, по сколько килограмм идёт мешок и каков выход кубов с одного мешка бетонной смеси. Вся эта арифметика раньше была раскидана по коду в двух проектах, и в этом, собственно, и было основное зло.

Когда спеки стали единым артефактом, дальше началось самое интересное: кодогенерация.

В репе сайта лежит небольшой TypeScript-скрипт, задача которого — прочитать все JSON-спеки, упаковать их в огромный Dart-овский `const Map`, и положить этот файл прямо в репозиторий Flutter-приложения как сгенерированный артефакт. Файл помечен как `generated`, редактировать руками его запрещено (линтер Flutter-а про это знает), и при каждой сборке приложения он берётся в том виде, в котором его оставил последний прогон скрипта. То есть цикл выглядит так: я правлю JSON на сайте, запускаю `npm run sync:specs`, коммичу изменение сразу и в репу сайта, и в репу приложения и оба проекта уезжают в прод с идентичными коэффициентами. Разойтись негде просто технически.

Но одной кодогенерации мне было мало. Потому что можно сгенерировать правильные константы и при этом каждый рантайм умудрится применить их по-своему. Код в TypeScript и код в Dart похожи, но не идентичны; где-то в TypeScript используется `Math.ceil`, где-то в Dart это `ceil()` на `num`, и теоретически есть ненулевой шанс, что на граничных значениях они поведут себя чуть-чуть по-разному. А «чуть-чуть по-разному» — это ровно тот случай, когда пользователь приходит с профлистом и 0,04 против 0,055.

Поэтому сверху легла вторая страховка: паритетные тесты. Механика такая. Есть набор фикстур — десятки комбинаций входных данных для каждого из шестидесяти одного калькулятора. Скрипт прогоняет TypeScript-движок по всем фикстурам и сохраняет ожидаемые результаты в отдельный файл. Ровно этот же файл лежит в Flutter-репе и гоняется там Dart-овским тест-раннером: тот же калькулятор, те же входы, проверка, что выход совпадает до последнего знака. Любое расхождение между двумя рантаймами сразу красный тест, красный CI, релиз не уезжает ни в прод сайта, ни в прод приложения.

Это звучит как перебор для пет-проекта, и, возможно, так и есть. Но по факту паритетные тесты за полгода поймали три реальных расхождения, не драматических, но вполне заметных. И каждый раз это происходило в момент, когда я полностью забыл, что менял логику. Именно за этим инвариантом и нужны машины.

Три режима точности, которых нигде нет

Пока я был глубоко в формулах, всплыл отдельный, куда более концептуальный вопрос: а какой вообще «правильный» ответ?

Классический онлайн-калькулятор считает так: взял норматив, накинул «запас 10%», выдал число. И вот это универсальное «плюс десять процентов на всё подряд» основной источник вранья в жанре. Плитке нужно заложить больше на подрезку в углах, но этого же запаса не надо для

прямого шва метро-укладки. Штукатурке нужно закладывать неровность стены, но если маяки уже стоят, неровность снимается ими, а не запасом материала. Клей и грунт ведут себя по-разному на разных основаниях. Один коэффициент на все случаи — это удобно для кода и вредно для пользователя.

Поэтому поверх чистых формул появился отдельный слой практических поправок с тремя режимами: базовый (нормативный расчёт, минимум поправок), реалистичный (дефолт, учитывает типичные условия ремонта) и профессиональный (осторожный режим, рассчитан на сложные условия и гарантированную закупку). Коэффициенты, опять же в JSON, опять же единый источник истины, опять же без пересборки.

И тут я добавил ещё одну самопроверку, на этот раз вообще не про формулы, а про здравый смысл. Есть жёсткое правило: чем осторожнее режим, тем больше материала. Базовый режим всегда меньше или равен реалистичному, реалистичный — меньше или равен профессиональному. На одинаковых входах это неравенство должно соблюдаться для любого калькулятора. В CI стоит отдельный тест, который это проверяет на всех шестидесяти одном. Если где-то коэффициент случайно крутанул не туда и «профессиональный» вдруг стал считать меньше, чем «базовый», как итог, падает сборка. Это такая бизнес-логическая капча против моих же опечаток.

Валидаторы поверх тестов

Раз уж заговорили про страховки, упомяну ещё три скрипта, которые живут отдельно от обычных unit-тестов и запускаются в CI параллельно. Они не проверяют, что конкретная формула на конкретном входе даёт конкретное число — это делает Vitest. Они проверяют общие инварианты, которые в системе должны соблюдаться всегда.

Один прогоняет все калькуляторы на синтетических входах и ловит сквозные нарушения: что количество к покупке всегда не меньше чистого количества, что результаты не уходят в нули при валидных входах, что единицы измерения в подписях всегда русские (не проскочило случайно m^2 вместо m^2), что инвариант между режимами точности соблюдается. Второй отдельно проверяет упаковки на реалистичность: клей не должен фасоваться в канистрах по 17 литров, плитка не продаётся по одной штуке. Третий ловит смешение систем единиц, то есть, если в каком-то калькуляторе миллиметры нечаянно сложились с метрами, он падает тут, а не в отзыве пользователя.

Это два разных уровня защиты. Unit-тесты — микроскоп, валидаторы — магнитная рамка на входе в аэропорт. На шестидесяти одном калькуляторе без рамки рано или поздно что-то протаскиваешь незаметно.

Михалыч переезжает на сервер

Теперь про ИИ. Михалыч — это тот самый ворчливый прораб, про которого я рассказывал в предыдущей статье. В приложении он изначально ходил в OpenRouter напрямую: у Dart-клиента был API-ключ, Dart-клиент формировал запрос, получал ответ, стримил в чат.

А потом была история, после которой я этот подход полностью перестроил. Это случилось недавно.

В какой-то момент я заметил, что расход токенов на OpenRouter растёт быстрее, чем растёт моя пользовательская база. Сначала я списывал это на сезонность возможно, просто подтягиваются новые пользователи, всё нормально. Но цифры не сходились. Разница была слишком большая, чтобы объясниться ростом аудитории. Кто-то решил побаловаться через мой ключ и воспользовался дырой в моём коде и поставить его себе, а дальше он просто использовать Orus 4.7 через Opencode и сожрал все деньги. Я полез разбираться.

 Claude Opus 4.7	OpenCode		758 032 → 675	\$ 4,63	Credits	26,8 tps	stop	 >
 Claude Opus 4.7	OpenCode		757 788 → 214	\$ 4,74	Credits	11,4 tps	stop	 >
 Claude Opus 4.7	OpenCode		752 994 → 851	\$ 0,411	Credits	55,0 tps	stop	 >
 Claude Opus 4.7	OpenCode		750 746 → 175	\$ 4,70	Credits	10,3 tps	tool_calls	 >
 Claude Opus 4.7	OpenCode		749 742 → 4 839	\$ 4,81	Credits	93,0 tps	tool_calls	 >
 Claude Sonnet 4.6	New API		8 → 16	\$ 0,000264	Credits	9,3 tps	length	 >
 Gemini 3 Flash Preview	Masterok		1 255 → 420	\$ 0,00189	Credits	104,7 tps	stop	 >
 Gemini 3 Flash Preview	Masterok		1 250 → 447	\$ 0,00197	Credits	103,6 tps	stop	 >

Opus 4.7 любит много кушать.

И тут выяснилась неприятная подробность из моего прошлого. API-ключ для OpenRouter был зашит в релизный билд приложения. Не как захардкоженная строка в исходниках, а «цивилизованно» через переменную окружения, которая подтягивалась на этапе сборки. Но итоговый результат на устройстве пользователя одинаковый: ключ лежит в APK, в открытом виде, в ресурсах. Никакой обфускации, никакой магии, просто строка на видном месте.

А APK в RuStore — это публичный артефакт. Любой желающий скачивает его себе на устройство, открывает любым декомпилятором и смотрит содержимое. Что, собственно, кто-то и сделал. Ключ вытащили, и им с удовольствием начали пользоваться на свои запросы. Я даже не могу на этого человека злиться: ключ лежал буквально на поверхности, найти его было тривиально, это чистая моя халатность.

Ротировал ключ, выдохнул, сел думать. И тут сработало то, что сайт и приложение это одна экосистема. У сайта уже был серверный API-роут `/api/mikhalych`, которым пользовался веб-чат. Я просто переключил приложение ходить туда же. Теперь приложение посылает запрос на свой собственный домен, сервер проверяет рейт-лимит, подставляет системный промпт, проксирует запрос в OpenRouter с ключом, который живёт только на сервере Timeweb, и стримит ответ обратно.

Ключа на устройстве пользователя физически больше нет. Даже если завтра кто-нибудь снова скачает APK и распотрошит его, он увидит только URL моего собственного эндпоинта, а эндпоинт прикрыт рейт-лимитом. Бонусом я получил возможность менять LLM-модель без релиза приложения: в прошлый раз переехал сначала с Gemini 3 flash на Claude Haiku 4.5, а затем Grok 4.1 Fast одной правкой переменной окружения, перезапустил контейнер, и оба интерфейса (веб и мобильный) поехали на новой модели. Это уровень контроля, который из приложения организовать в принципе невозможно.

Soft 404, из-за которого я переехал с SSG на SSR

Начинал сайт я на `output: "export"` в конфиге Next.js, в связке с Frontend App на Timeweb. Быстро, дёшево, раздача статики с CDN-like поведением. Одна проблема: Frontend App на любом несуществующем URL отдавал SPA-fallback на `index.html`. Статус 200 OK, контент главной страницы.

Для живого пользователя это выглядит относительно прилично: попал не туда, увидел знакомую главную, не понял что произошло, но продолжил листать. Для Google это выглядит совсем иначе. Это называется Soft 404: страница формально жива, но её контент не соответствует URL, значит, либо сайт сломан, либо кто-то пытается играть с индексом. И Google начал методично выкидывать мои страницы из индекса. Не все, но достаточно, чтобы я заметил падение в Search Console.

Лечение оказалось радикальным: переезд на Backend App с настоящим Node.js, настоящий `next start`, App Router, файл `not-found.tsx` с честным HTTP 404. После этого отсутствующие URL стали отдавать правильный статус, Google удовлетворённо кивнул и вернул обратно в индекс те страницы, которые успел выкинуть (ну, большую часть).

Мораль оказалась вот такая. Static export — не бесплатная оптимизация. Это режим, который перекладывает ответственность за роутинг на ваш хостинг, и если ваш хостинг умеет только SPA-fallback, то с точки зрения поисковика вы собственноручно объявили весь свой сайт обманом. Взрывается такая мина тихо, не по алерту, а по медленному падению позиций.

308-редирект, который сломал мобильное приложение

Включил `trailingSlash: true` в конфиге, чтобы все страницы индексировались единообразно (со слэшем на конце). Next.js исправно начал отдавать 308-й редирект с `/api/mikhalych` на `/api/mikhalych/`. Всё хорошо, сайт доволен, браузер доволен.

Недоволен оказался Dart.

Клиент `http.Client` в Dart не повторяет тело POST-запроса после 308-го редиректа. RFC говорит, что должен. Dart говорит, что не будет. Итог: приложение шлёт сообщение Михалычу, получает 308, послушно перепрыгивает на URL со слэшем, но уже без тела. Сервер получает пустой POST и отвечает молчанием. Пользователь видит пустой ответ в чате. Ни ошибки, ни исключения, ни таймаута — просто тишина.

Ловил я это полдня. Лечение в итоге распределилось на два файла. В конфиге Next.js я отключил встроенное поведение trailing slash для всего сайта. В ответ на это в middleware появился маленький кусочек логики, который возвращает 308 только для обычных страниц, а API-роуты проходят через `rewrite` без редиректа. Двадцать строк кода, которые починили то, что сломала одна строка в конфиге.

Я потом почитал трекер Dart, там эта проблема известная. Фикс, кажется, обсуждают. Но на момент моих приключений, не было.

PWA, которую пришлось выпиливать с мясом

Одна из ранних версий сайта была PWA — `manifest`, `service worker`, офлайн-режим, иконка на домашний экран. Звучит здорово. Работает отвратительно, если ты часто релизишь.

Service worker кэширует старые JS-бандлы. Пользователь открывает сайт через месяц, а получает калькулятор, который считает по-старому. Я релизил фикс. Пользователь открывал сайт ещё через неделю, получал тот же самый свой закэшированный калькулятор, который считает по-старому. Кто-то приходил жаловаться, я просил сделать жёсткий рефреш, кто-то делал, кто-то нет. Большинство просто уходило в другую вкладку и не возвращалось.

Я выпилил PWA. Удалил манифест, удалил `service worker`, удалил всю сопутствующую инфраструктуру.

И тут поймал следующее веселье: у всех, кто уже хоть раз открыл сайт, `service worker` никуда не делся. Он живёт в браузере, перехватывает запросы и отдаёт свою кэш-версию. Сам по себе он не умрёт, нужно, чтобы кто-то пришёл и явно его снёс. Пришлось добавить в глобальный layout маленький клиентский скрипт, который при загрузке страницы проверяет наличие зарегистрированных `service worker`ов, аккуратно их вычищает и заодно подчищает старые кэши. Скрипт прожил в проекте пару месяцев и тут я прикинул, что все мало-мальски активные пользователи за это время должны были зайти хоть раз. После этого удалил и его.

Осадочек остался. PWA — это не просто галочка в манифесте, это контракт, который ты подписал с браузерами пользователей. Разорвать его в одностороннем порядке можно, но стоит это осторожной санации и нескольких месяцев ожидания.

Ghost как CMS, цены у пользователя и выдуманный эксперт

Блог на сайте живёт на Ghost, но не в том смысле, в котором живут большинство Ghost-блогов. Ghost у меня стоит не сайтом, а редакторской админкой: весь контент пишется и хранится там, а отображается он через Next.js через Content API. Сырой HTML из Ghost прогоняется через DOMPurify (иначе можно словить XSS из собственного блога — звучит как оксюморон, но мы все знаем, как это бывает).

Простой и удобный редактор с отличным интерфейсом

Ghost — одна из лучших прозаических админок, которые мне попадались. Удобный редактор, автосохранение, нормальная работа с картинками, Admin API, через который можно пачкой обновить метаданные во всех постах одной командой. Когда блог ведёт один человек и постов в нём двадцать это важнее, чем вся та «богатая функциональность», которой пугают корпоративные CMS.

Следом история про цены, которая по своей природе похожа на историю про формулы.

В калькуляторе «стоимость ремонта» у меня одно время лежали пятьдесят восемь хардкод-цен: штукатурка по такой-то цене за мешок, шпаклёвка по такой-то, плитка такая-то. Проблема очевидная: цены в России в 2026 году меняются быстро. Я поддерживал эти цены ровно до того момента, пока первый пользователь не написал «а у меня этот материал стоит в полтора раза дороже, у вас ерунда».

И он прав. Это моя ерунда. Я физически не могу честно поддерживать пятьдесят восемь актуальных цен по всей России. Можно конечно парсить цены с разных городов, но это не нужно.

Решение оказалось в духе всего остального сайта: цены принадлежат пользователю, а не мне. Есть небольшая утилита, которая хранит пользовательские цены в localStorage, подтягивает их во все калькуляторы автоматически и показывает специальный значок рядом с теми ценами, которые пользователь переопределил. Один раз ввёл свои локальные цены, а дальше все расчёты идут по ним. Не понравилось — сбрасывается одной кнопкой. Сайт перестал врать, а я перестал

чувствовать себя самозванцем, поддерживающим актуальность того, что актуализировать не в моих силах.

И в ту же рубрику «честно вместо красиво» ещё одна история про страницу «О специалисте». Была такая страница, и был на ней выдуманный эксперт-прораб. Мифический Иван Михайлович с тридцатилетним стажем, в честь которого, собственно, Михалыча и назвал. И в какой-то момент я сам себя спросил: а что я отвечу, если пользователь напишет «дайте ссылки на публикации вашего эксперта»? Ответ был только один, молчать и удалять страницу. Я и удалил.

Доверие, как выяснилось, не падает от того, что ты не врёшь. Оно падает от того, что ты врёшь и попадаешься.

SEO для российского рынка

Сайт русскоязычный, ориентирован на рынок РФ, и это определяет целый букет решений, которые в европейском или американском пет-проекте выглядели бы странно.

Аналитика — Яндекс.Метрика, не Google Analytics (статистику пока отслеживаю в GSC. У Метрики при этом есть вебвизор, удобная фишка, это буквально запись сессии пользователя, где видно, что он жмёт, где зависает и где в итоге уходит. Для отладки UX это незаменимо, ни у GA, ни у Plausible ничего подобного из коробки нет.

Хостинг — Timeweb Cloud, не Vercel и не AWS. Оплата российской картой, поддержка по-русски, центры обработки данных в Москве и Петербурге, никаких танцев с санкциями и платежами через третьи страны. Android-дистрибуция — RuStore, не Google Play. Play Console из РФ, отдельный клубок боли, и мне было проще заплатить за эту боль тем, что я теряю часть мирового рынка, чем разбираться с зарубежной картой и верификацией (попробовал - не понравилось).

Из SEO-мелочей, которые на самом деле не мелочи. Sitemap у меня не статичный файл, а Next.js-роут, который при каждой сборке обходит все калькуляторы, инструменты, чек-листы и посты блога, предоставляет каждой странице градуированный приоритет (главная, хабы, популярные калькуляторы, обычные калькуляторы, блог) и собирает итоговую карту в памяти. После каждого билда в postbuild автоматически идёт пинг в сервис оповещения поисковиков об обновлённом sitemap. На страницах калькуляторов лежит JSON-LD разметка: `BreadcrumbList` и `QAPage` для FAQ-блоков. Раньше ещё был `AggregateRating`, но я его снёс, ведь настоящих отзывов у меня не было, а фейковые рейтинги Яндекс научился ловить и отправлять сайт в пессимизацию. Лучше никакого рейтинга, чем рейтинг, за который можно получить по рукам.

А самый свежий кусок — это `/llms.txt`. Молодой стандарт, аналог `robots.txt`, только для LLM-краулеров: ChatGPT, Perplexity, Claude, GigaChat. Идея простая, вместо того, чтобы нейросеть выдирала из вашей страницы случайные куски текста, вы даёте ей аккуратный карманный гайд, что у вас за сайт, что на нём есть, какие основные разделы. Я отдаю такой файл по `getmasterok.ru/llms.txt`: короткое описание, список категорий, топ-15 калькуляторов по популярности, последние двадцать постов блога, строчка про лицензию. Файл генерируется динамически из тех же источников, что и sitemap, так что устаревать он не умеет. Стандарт ещё молодой, большая часть краулеров только начинает его уважать. Но когда начнут, я уже там. Лучше положить файл, который через полгода станет бесплатным каналом, чем через полгода спохватываться.

Про рекламу

Теперь, пожалуй, самый скучный раздел статьи, но без него картина будет неполная.

На сайте нет рекламы. От слова совсем. Ни баннеров, ни РСЯ, ни нативных блоков, ни всплывающих окон, ни встроенных в контент рекомендаций. В ближайшее время и не появится. И

дело тут не в каком-то принципиальном манифесте против рекламы, сам я понимаю, что она нужна, и чужие сайты со встроенной рекламой я использую регулярно. Дело в элементарной математике.

На нынешнем трафике любой баннер не окупит даже чашку кофе в месяц. Зато он гарантированно ухудшит пользовательский опыт: съест половину экрана на мобильном, замедлит загрузку, встанет между пользователем и формой ввода. Пользователь уйдёт, метрика показателей отказов поползёт вверх, поведенческие факторы в выдаче просядут, позиции в индексе ухудшатся, трафика станет меньше, и в следующем месяце тот же самый баннер заработает ещё меньше. Я видел сайты, которые по этой траектории скатываются до состояния «страница 80% реклама, 20% контент», и мне не хочется туда.

В долгосрочной перспективе монетизация на сайте, конечно, будет. Но я больше склоняюсь к модели подписки, а не баннеров. Какой-то набор базовых калькуляторов остаётся бесплатным навсегда, потому что это ядро, ради которого человек пришёл, и отжимать с него деньги за пересчёт пакетов плитки мне кажется занятием сомнительным. А сверху могут быть платные вещи: расширенная аналитика проектов, пачка ИИ-запросов к Михалычу без лимита, выгрузка красиво свёрстанной сметы, синхронизация расчётов между устройствами. Вот за такое люди платить готовы, и это не портит опыт для тех, кто просто зашёл посчитать двести мешков наливного пола. Может быть, добавлю и какую-то минимальную аккуратную рекламу, но точно не баннерные простыни, к которым привыкли крупные строительные сайты. Пока до этого всего далеко, и пока я могу себе позволить сайт без рекламы — сайт будет без рекламы.

Итоги

Если всё, что выше, свернуть в одну мысль, она будет такая: сайт — это не «Next.js-проект», это источник истины, к которому пристёгнуты два UI-рантайма, три слоя тестов и один ворчливый прораб на сервере.

Главное, что я вынес из всей этой истории — это, наверное, банальная, но больно выстрадавшая мысль, что как только у вас появляется больше одного клиента на одну и ту же бизнес-логику, всё, что не код, нужно немедленно выносить наружу. В JSON, в YAML, в базу, куда угодно, лишь бы это не было разбросано по двум кодовым базам на двух разных языках. Из вынесенного JSON дальше генерируется то, что нельзя прочитать на лету, и читается на лету то, что нельзя сгенерировать. Два рантайма, одна истина. И сверху обязательно паритетные тесты, которые проверяют, что рантаймы действительно считают одинаково. Без них вы узнаете о расхождениях от пользователей, и это дорогой способ.

Второе, что ключи API в клиенте нет никакого способа спрятать. Даже если у вас не Flutter, а обфусцированный нативный билд. Даже если кажется, что никто не будет его разбирать. Разберут, вытащат, будут пользоваться, пока не закончится баланс или пока вы не заметите. Единственный надёжный способ, ключ живёт только на сервере, а клиент ходит через ваш собственный прокси. Бонусом получаете рейт-лимит, логи и возможность менять LLM-модель без релиза приложения.

И третье — иногда самые интересные инженерные истории начинаются не со сложной архитектурной задачи, а с одного сообщения в личке на две строчки. У меня было сообщение про 0,015 м³ бетона. У вас, возможно, будет своё.

Сайт, про который всё это — getmasterok.ru. 61 калькулятор, бесплатно, без регистрации, без рекламы. Можно зайти, посчитать свою плитку, написать мне, если нашли у меня нестыковку с приложением. Теперь я хотя бы знаю, что с этим делать. Я не хвастаюсь своим сайтом, знаю, что он далеко не идеальный и ещё ему долго расти, но знаю, что по факту у меня есть некая своя маленькая экосистема из нескольких продуктов и это здорово :). Обязательно пишите фидбек если решите воспользоваться расчётами.

А приложение — то самое «Мастерок» из [прошлой статьи](#), в RuStore. Считает по тем же JSON-ам.

Теги: сайтостроение, приложение для android, next.js, timeweb, node.js фреймворки, строительный калькулятор, калькулятор, калькулятор стоимости, ai, ai-агенты

Хабы: Dart, Flutter, HTML, JavaScript, Node.JS

 +1 

 4



 0

Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Подписаться

Оставляя почту, я принимаю [Политику конфиденциальности](#) и даю согласие на получение рассылок



4K+

Охват за 30 дней

6

Карма

Soft Gaming @Soft_gAming
Пользователь

Подписаться



Поток Бэкенд доступен 24/7 благодаря поддержке друзей Хабра

Хабр Курсы для бэкендеров
Практикум, Хекслет, SkyPro, авторские курсы — собрали всех и попросили скидки. Осталось выбрать!

Перейти

РЕКЛАМА

[Перейти в поток Бэкенд](#)

 [Комментировать](#)

Публикации

[ЛУЧШИЕ ЗА СУТКИ](#) [ПОХОЖИЕ](#)



the_bat
23 часа назад

Ремонт блока питания с Power Delivery. 470 граммов электроники

 Простой

 4 мин

 16К

 +97

 31

 27



interpres
18 часов назад

Электроинструмент становится хуже, и это делается намеренно

 Простой  11 мин  27K

[Обзор](#) [Перевод](#)

 +72

 45

 85



prohronus

19 часов назад

Как ИИ-агенты стали новым оружием скамеров на Хабр Карьере

 5 мин  9.3K

[Кейс](#)

 +52

 9

 9



alizar

22 часа назад

Готовимся к отключению. Эффективные форматы для упаковки и раздачи HTML-страниц

 Простой  7 мин  12K

[Обзор](#)

 +37

 59

 18



AlexSerbul

23 часа назад

Программирование с AI-ассистентом — похороните меня под плинтусом

 Средний  14 мин  10K

[Мнение](#)

 +28

 33

 11



TrexSelectel

19 часов назад

Linux 7.0 и что изменилось в ядре после очередного цикла разработки

 5 мин  8.8K

[Обзор](#)

 +27

 10

 0



mariklolik

19 часов назад

Как переложить нагрузку по code review с разработчиков на LLM

 Средний  7 мин  7.5K

[Кейс](#)

 +23

 23

 6



nick_dyuba

23 часа назад

Легенды 90-х — кто придумал и производил жвачки Turbo, Love is... и TіріTіr

🕒 5 мин 👁 7.1K

Recovery Mode

📦 +22

🔖 8

💬 5



Mimizavr

23 часа назад

«Великое очищение» в работе с контентом: что осталось от роли редактора

🕒 11 мин 👁 7K

📦 +17

🔖 9

💬 7



infgotoinf

11 часов назад

Perl — зря забытый язык программирования?

🕒 11 мин 👁 8.6K

Из песочницы

📦 +16

🔖 13

💬 10

Big Data в фарме: как мы управляем сотнями сотрудников по всей стране

Турбо

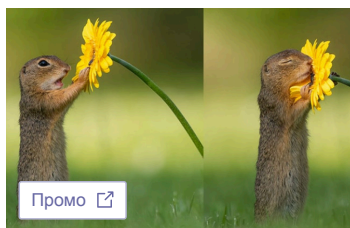
Показать еще

МИНУТОЧКУ ВНИМАНИЯ



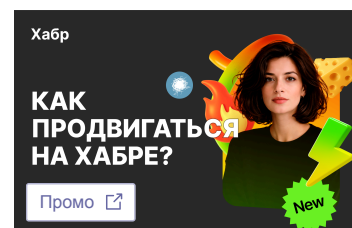
Турбо

Удалёнка в фарме: большие данные и никакой магии



Промо

Запахло весной и скидками в Промокодусе



Промо

Рассказываем в обновленном и расширенном курсе

ВОПРОСЫ И ОТВЕТЫ

Как поменять загрузочный диск в Linux?

Linux · Простой · 2 ответа

Сайт на виртуальном хостинг Timeweb с размером 107,93 КБ даёт ответ сервера 73 мс. Что делать?

Timeweb · Простой · 2 ответа

Как задеплоить vite react сайт на хостинг timeweb?

Timeweb · Простой · 2 ответа

Почему не работают cookie на хостинге?

PHP · Средний · 1 ответ

Можно ли по умолчанию настроить сервер так, чтоб домен запускал страницу django, а не страницу веб-сервера apache?

Django · Простой · 2 ответа

[Больше вопросов на Хабр Q&A](#)

ЧИТАЮТ СЕЙЧАС

Электроинструмент становится хуже, и это делается намеренно

 27K  85

Тим Кук покидает пост Apple. Новый глава — «отец» Apple Silicon Джон Тернус

 14K  15

Представлен проект KillerPDF — редактор PDF с открытым исходным кодом для Windows 10/11

 7.2K  16

Пользователь купил Atomic Heart на VK Play, но не смог запустить игру из-за сетевых проблем

 5.7K  12

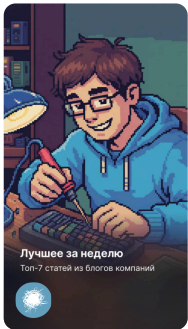
+185% за 13 часов: как Kimi K2.6 переписала 8-летний движок

 14K  11

Big Data в фарме: как мы управляем сотнями сотрудников по всей стране

Турбо

ИСТОРИИ



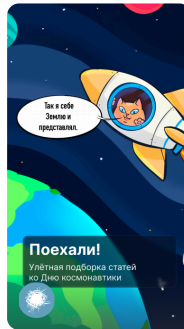
Годнота из блогов компаний



Только 10% решат эту головоломку



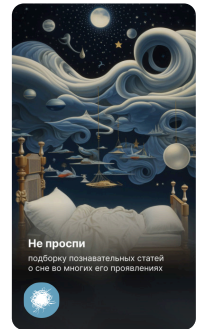
Хочу 450К



Вперёд к звёздам



Там на неведомых дорожках...



Самое время поспать

БЛИЖАЙШИЕ СОБЫТИЯ



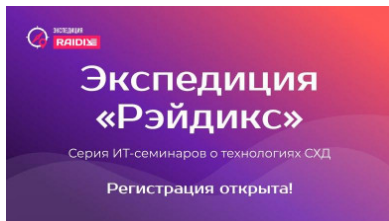
2 марта – 10 августа

**PROBA Awards —
международная
коммуникационная
премия, учрежденная в
2000 году**

Санкт-Петербург

Маркетинг

[Больше событий в календаре](#)



3 марта – 9 июня

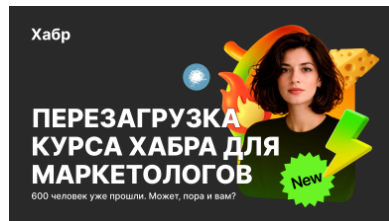
Экспедиция «Рэйдикс»

Екатеринбург • Новосибирск •
Краснодар • Минск • Калининград •
Ростов-на-Дону

Разработка

Другое

[Больше событий в календаре](#)



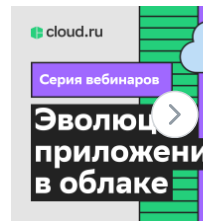
9 марта – 1 июня

**Всё, что нужно, чтобы
стартовать продвижение
бренда на Хабре**

Онлайн

Маркетинг

[Больше событий в календаре](#)



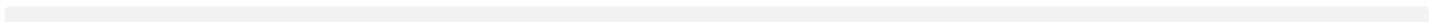
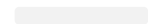
19 марта – 28 апреля

**Серия вебинаров
«Эволюция п
в облаке»**

Онлайн

Разработка

[Больше событий в календаре](#)



Ваш аккаунт

Войти

Регистрация

Разделы

Статьи

Новости

Хабы

Компании

Авторы

Песочница

Информация

Устройство сайта

Для авторов

Для компаний

Документы

Соглашение

Конфиденциальность

Услуги

Корпоративный блог

Медийная реклама

Нативные проекты

Образовательные программы

Стартапам



Настройка языка

Техническая поддержка

© 2006–2026, Habr